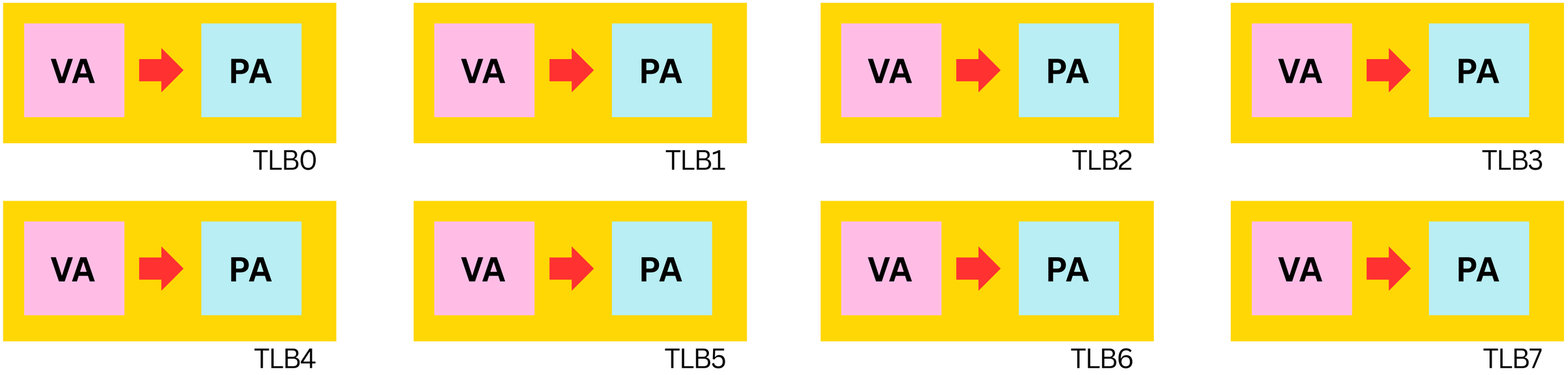
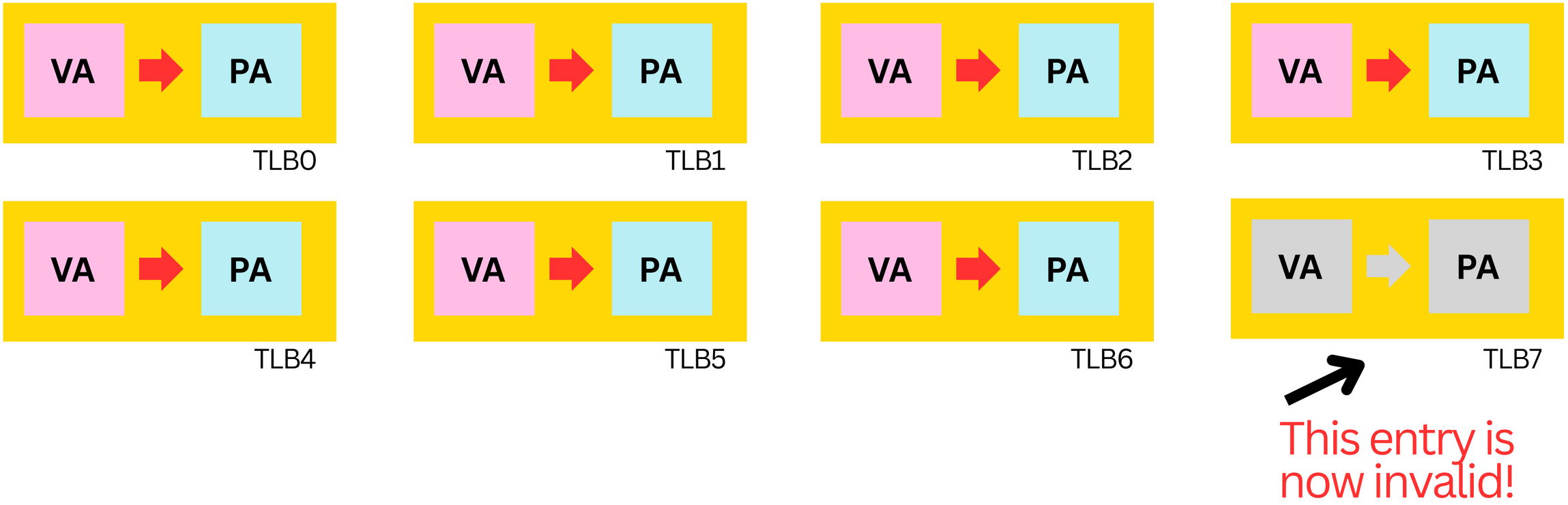
Page Tables

Using the TLB to support threads with virtual memory is a viable approach, but we need something else if we want to scale.

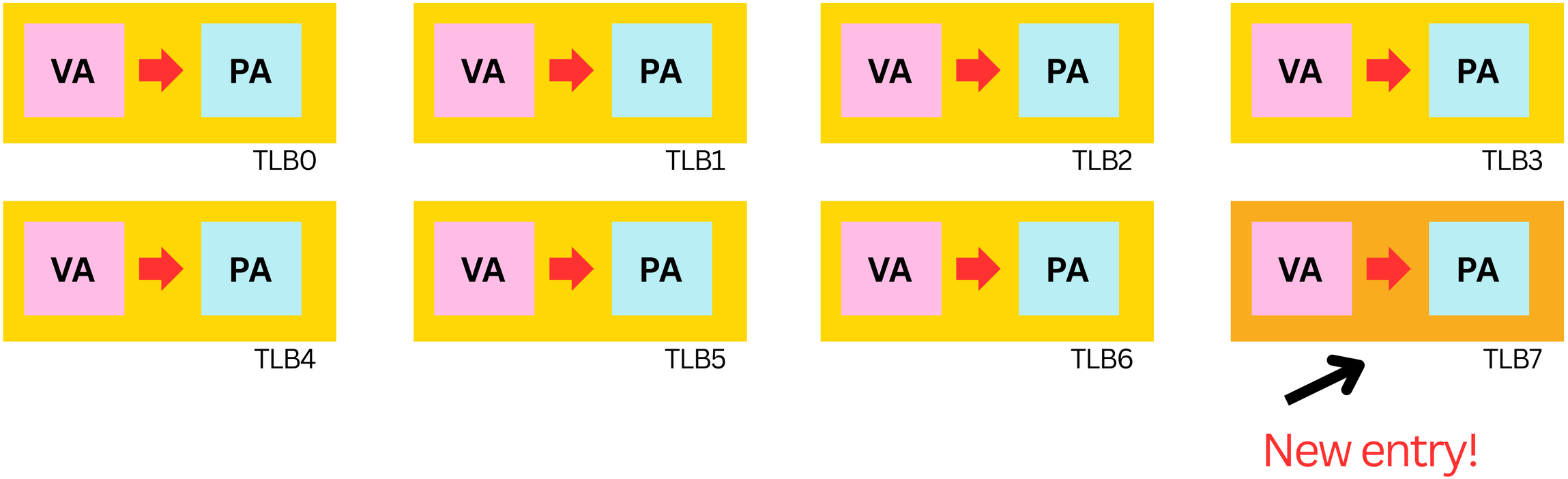
What if we have **many** threads running at the same time?



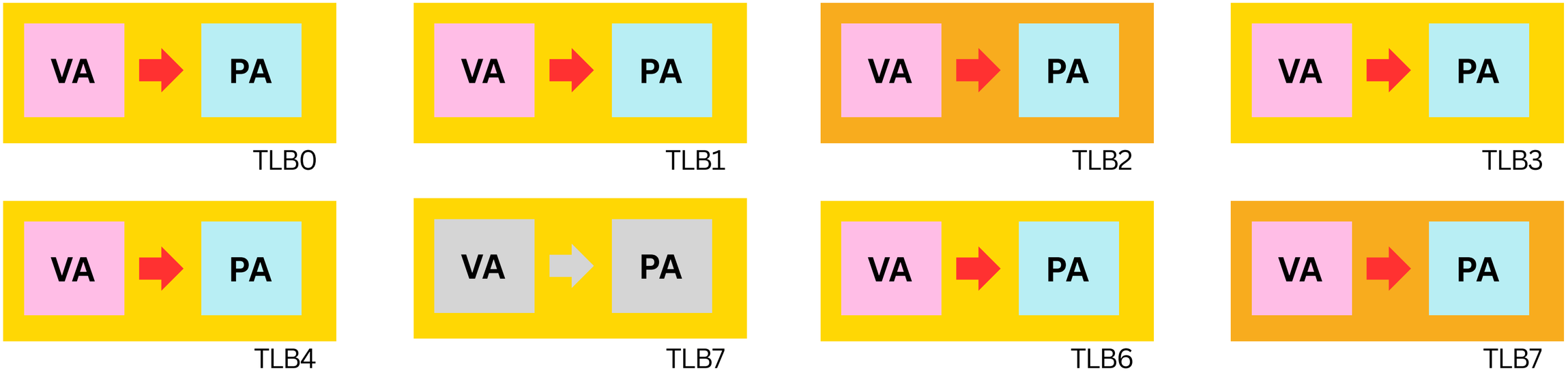
We'll have to keep swapping mappings out...



And manually replace with the new one...



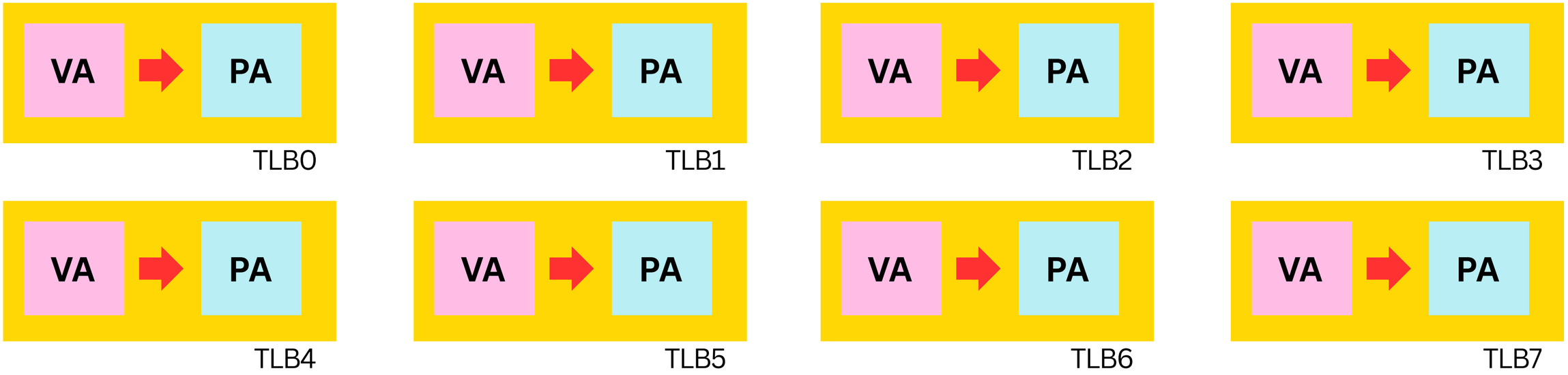
This frequent swapping can cause a lot of unnecessary performance overhead.



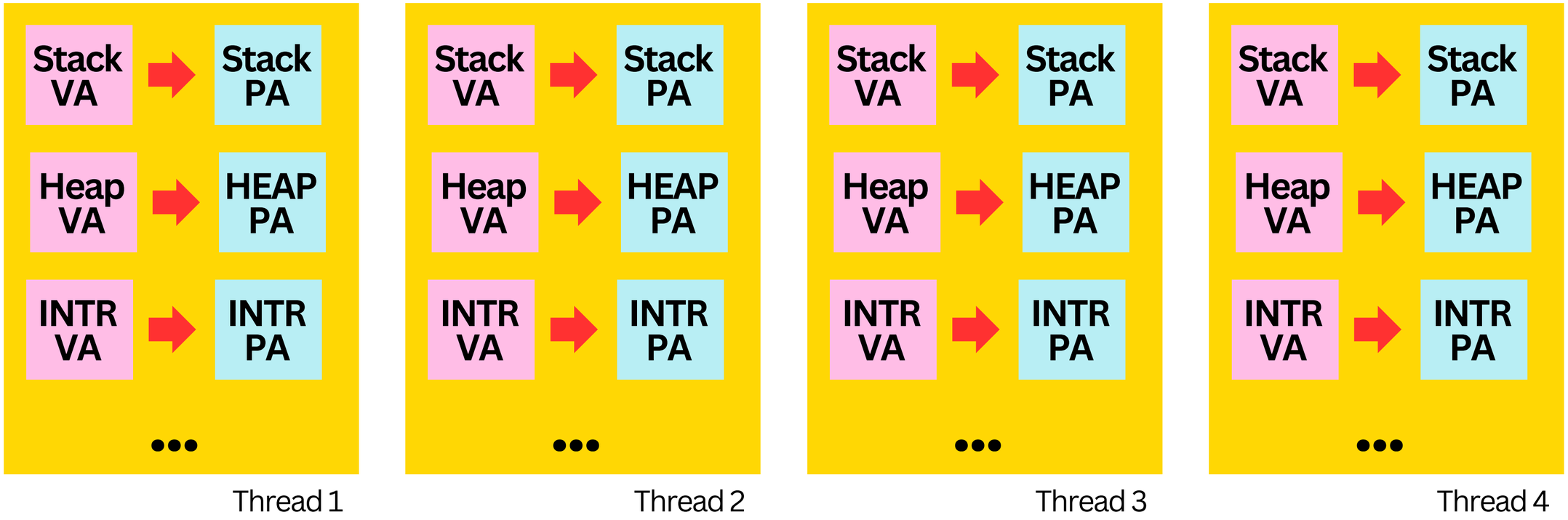
Solution: page tables

- Allows each thread to have a dedicated virtual memory mapping.
- Prevents one threads memory layout from interfering with another.
- MUCH more space than just 8 entries -- each thread can have many more associated mappings.

So instead of this for just a few threads...



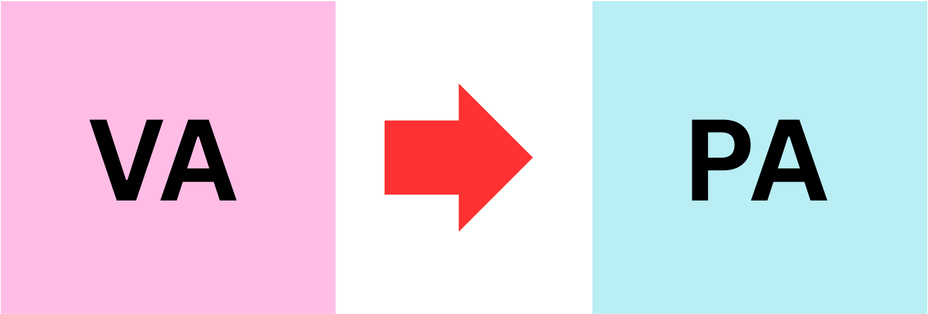
We can have many more mappings per thread:



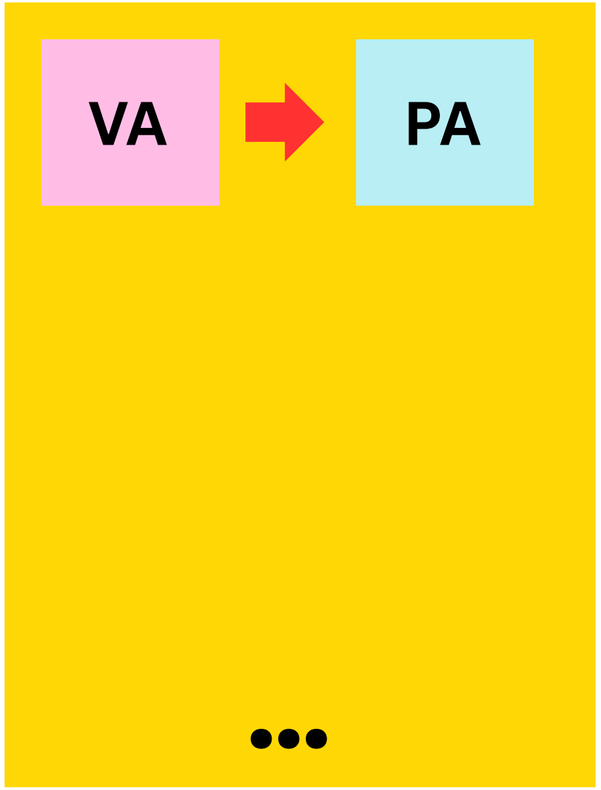
•••

(And so on)

Remember our goal -- virtual memory maps a virtual address to a physical address

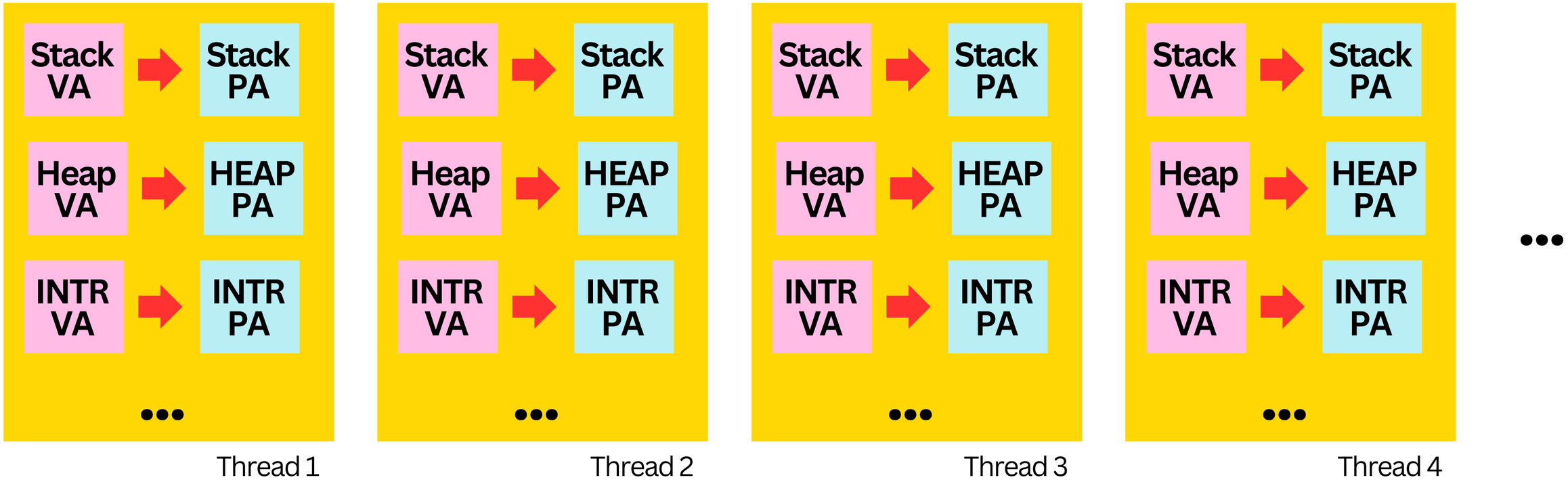


In a page table, it's similar to the TLB to create these mappings -- just pin it in the thread-specific page table

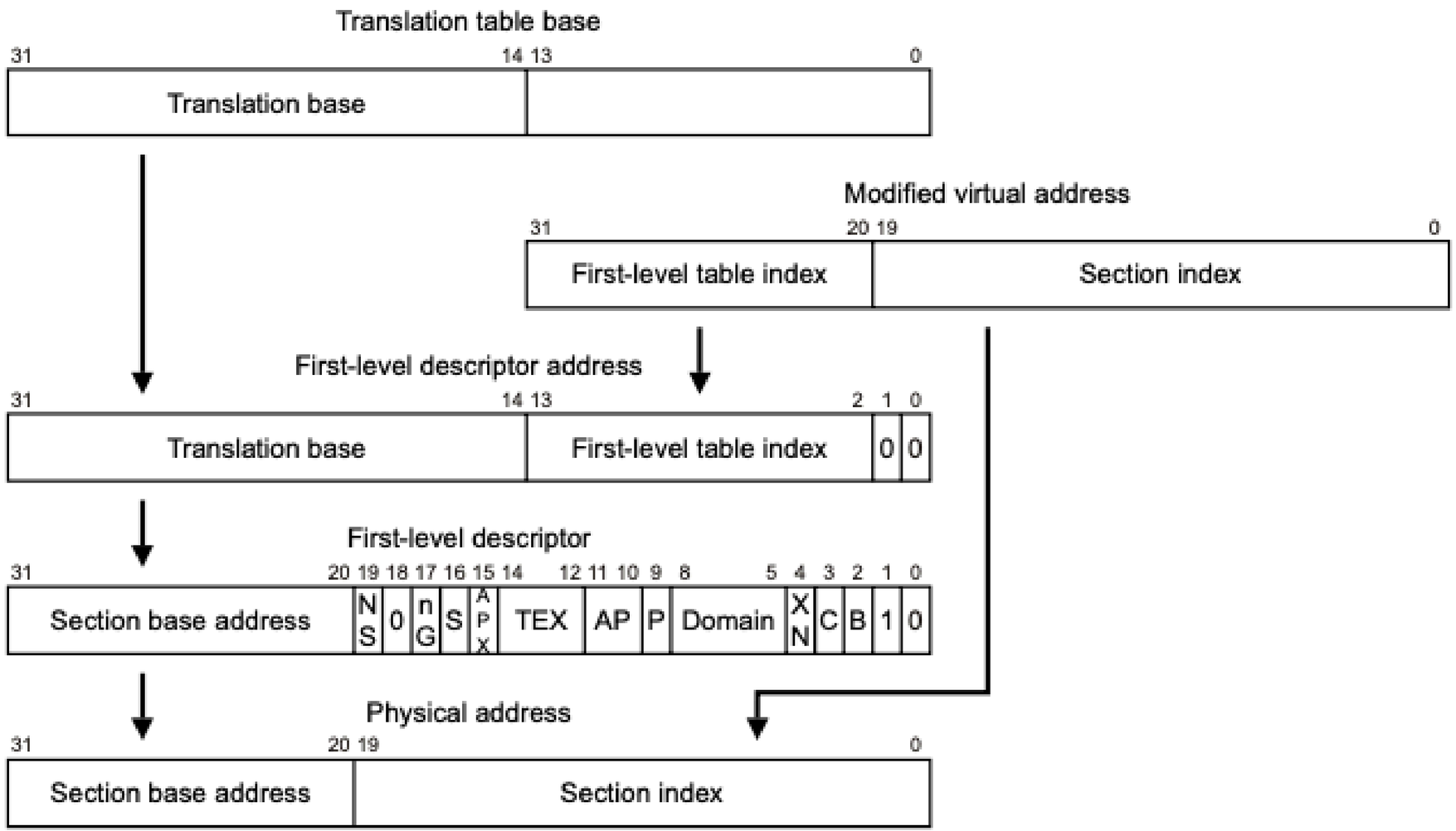


Thread 1

The tricky part is **finding** the desired page table and **entry** within it after many have been created.

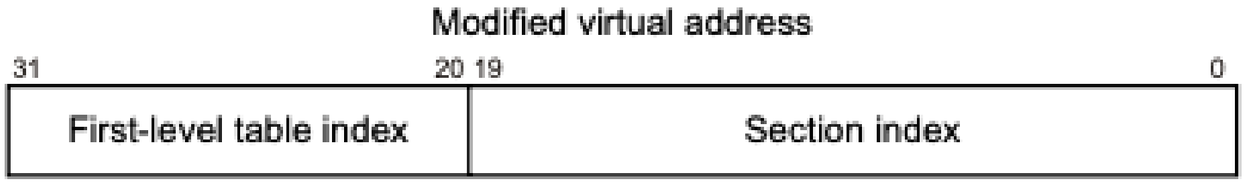


The key is using the **first level descriptor translation** process (pg **6-46** of the ARM manual).

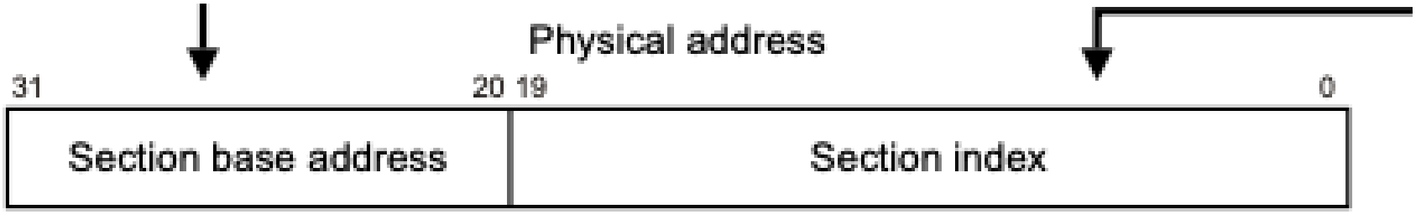


Let's break this down...

These instructions will allow us to get from the **virtual address**...

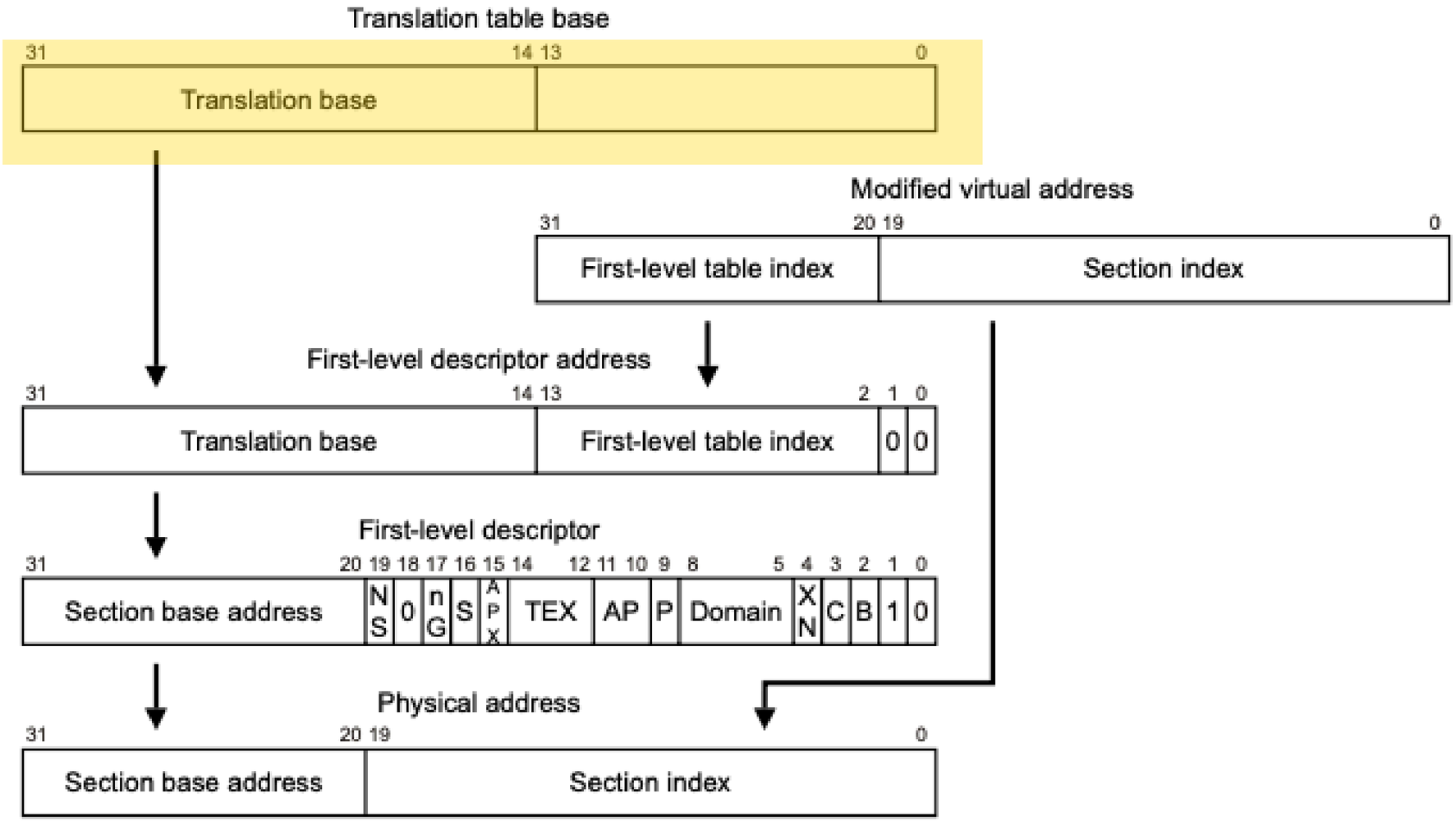


(some magic we'll explain soon)

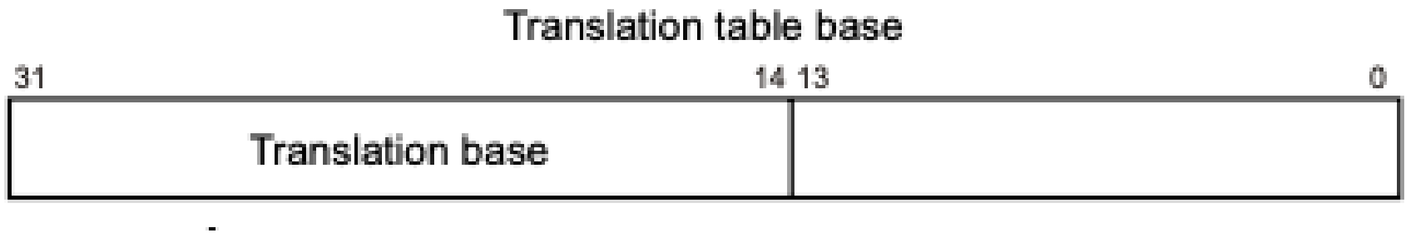


to the **physical address**.

First: the **translation table base register (TTBR0)**

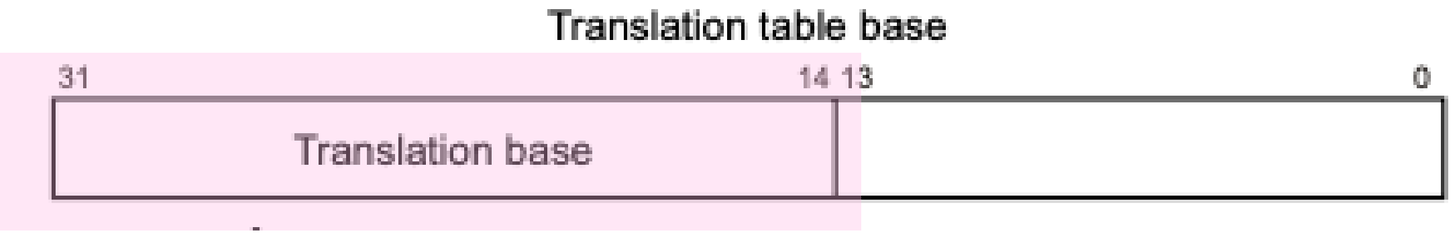


The TTBR0 (don’t worry about TTBR1 for this lab) holds the **address** of the currently **active page table** in memory.

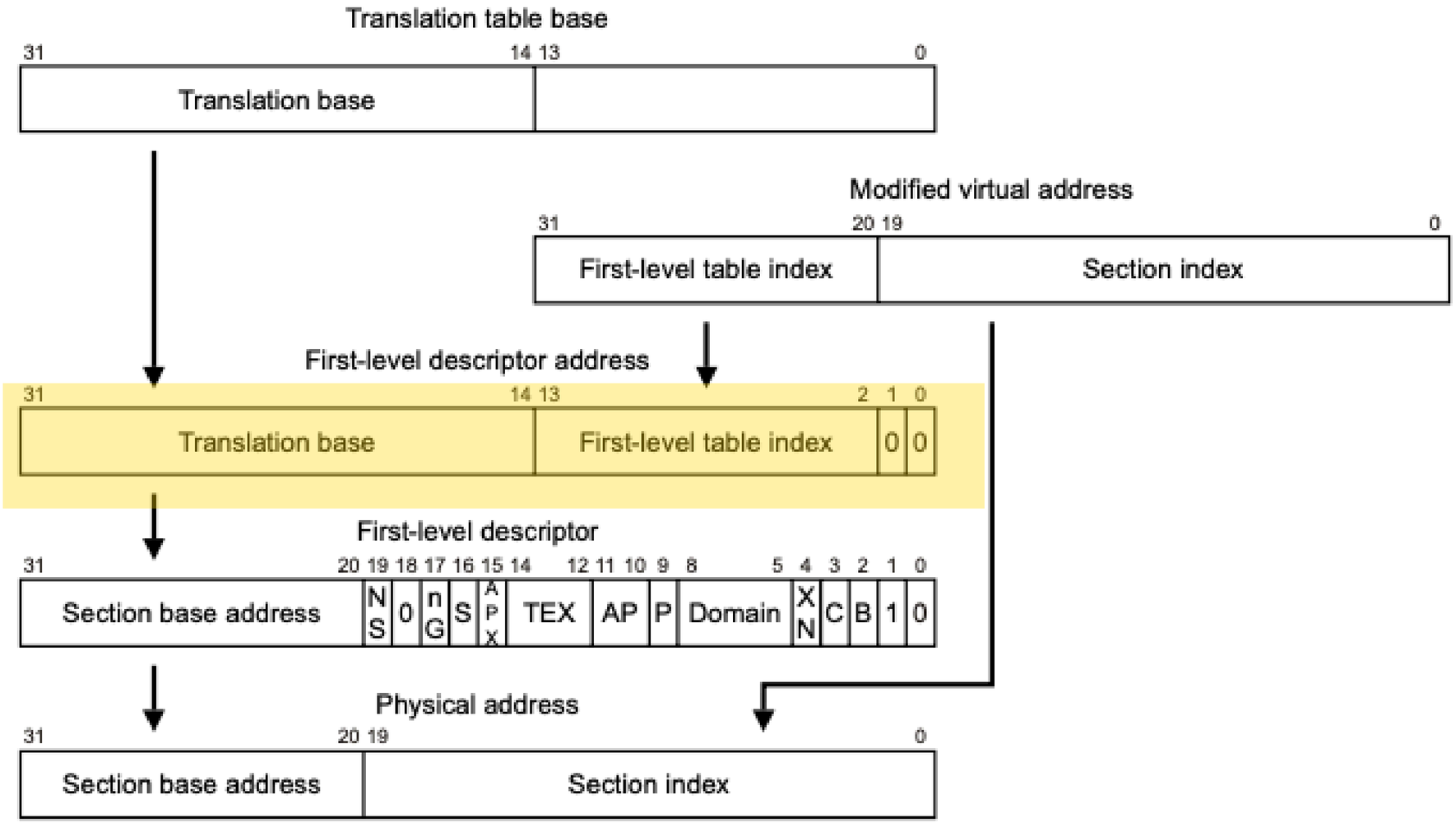


Be sure that the thread you’re currently considering has its **page table stored in this register**, so consider when you’ll need to (context) switch.

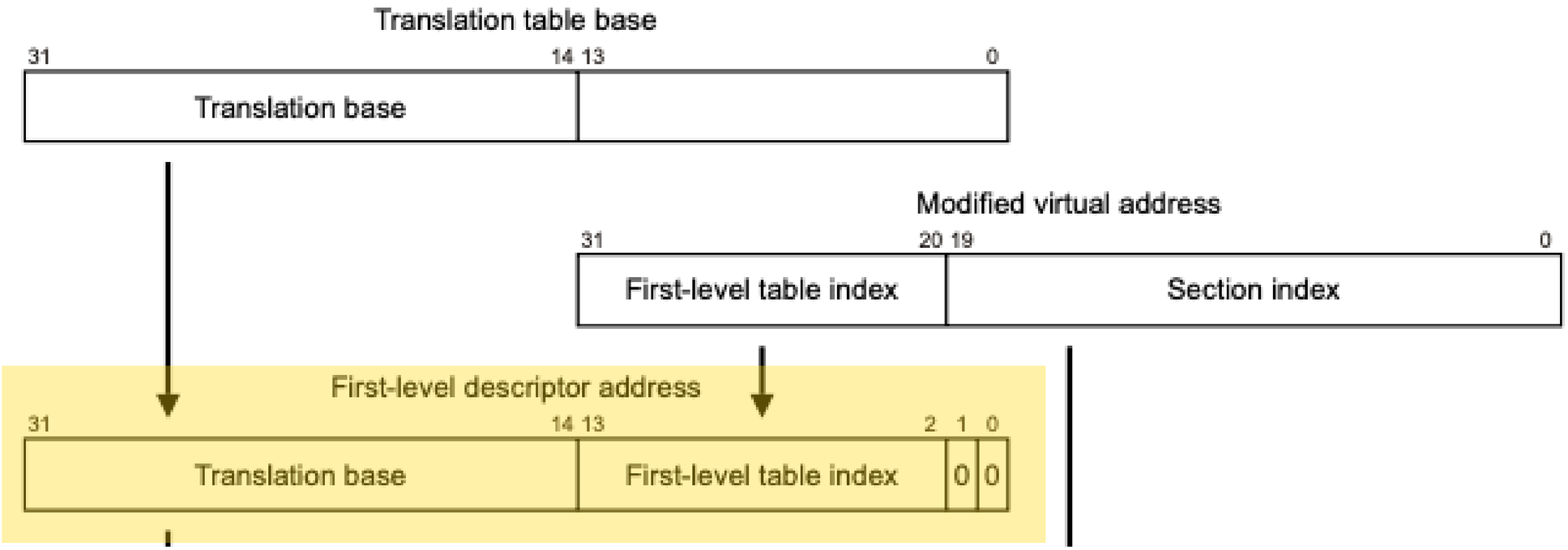
We'll use bits 31:14 to (partially) help determine where the **entry** is in the page table... so hold on to this!



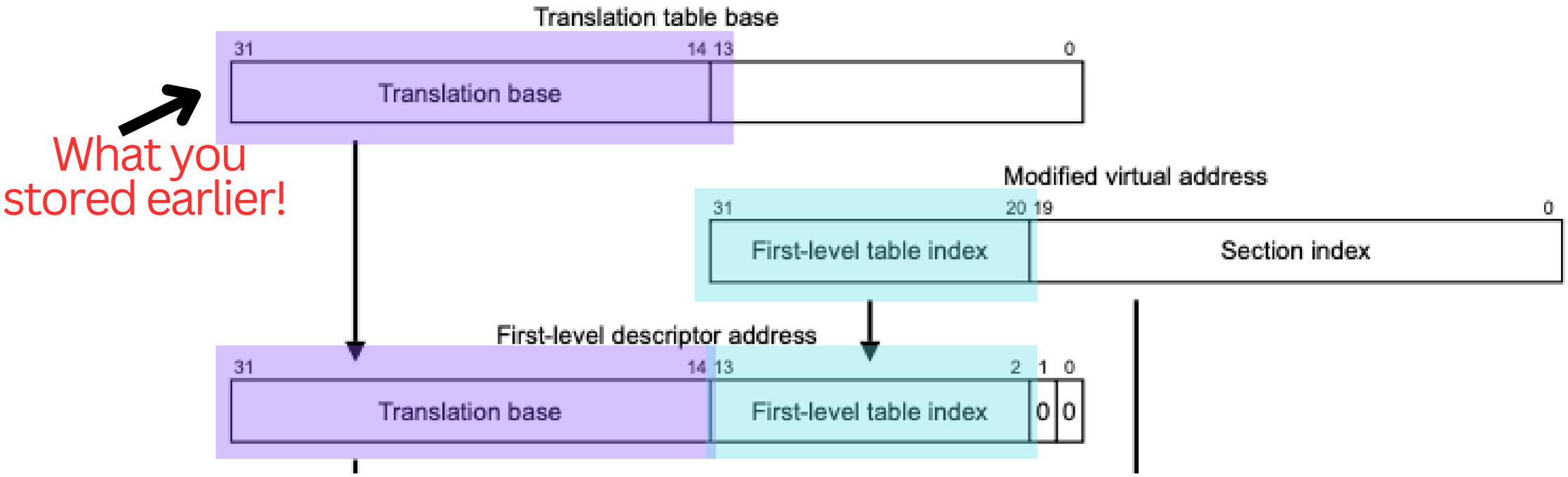
Next: the **first level descriptor address**



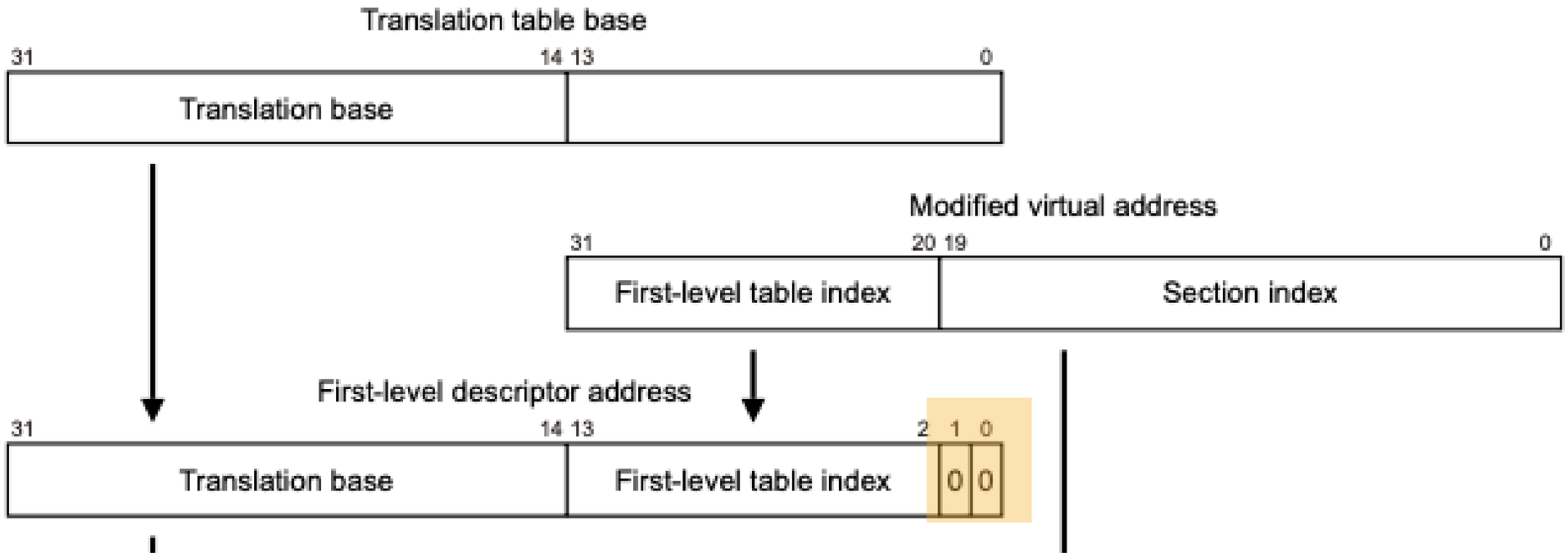
The first level descriptor address holds the address of the specific **entry** in the page table.



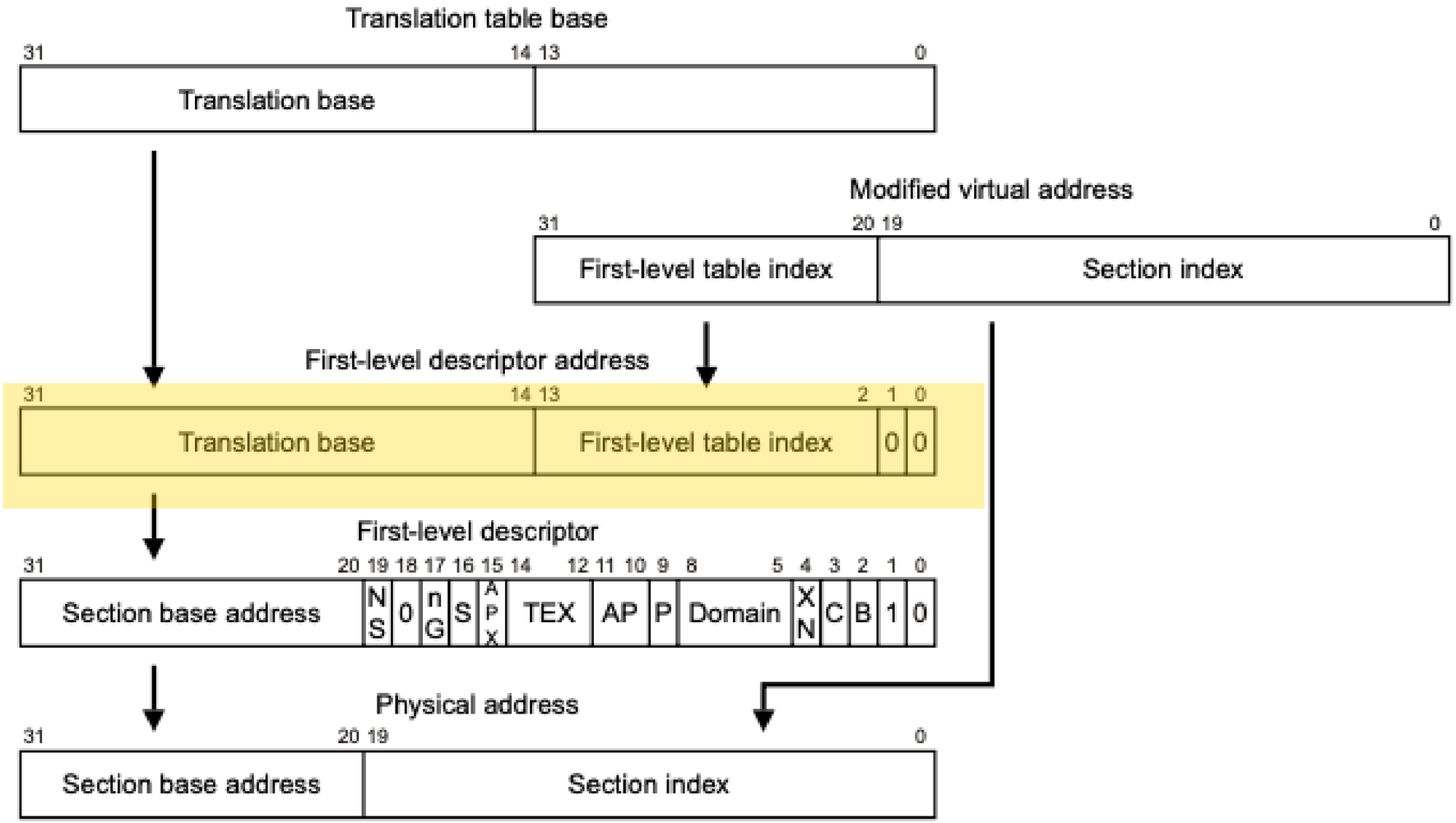
It requires information from both the TTBR0 and the virtual address.



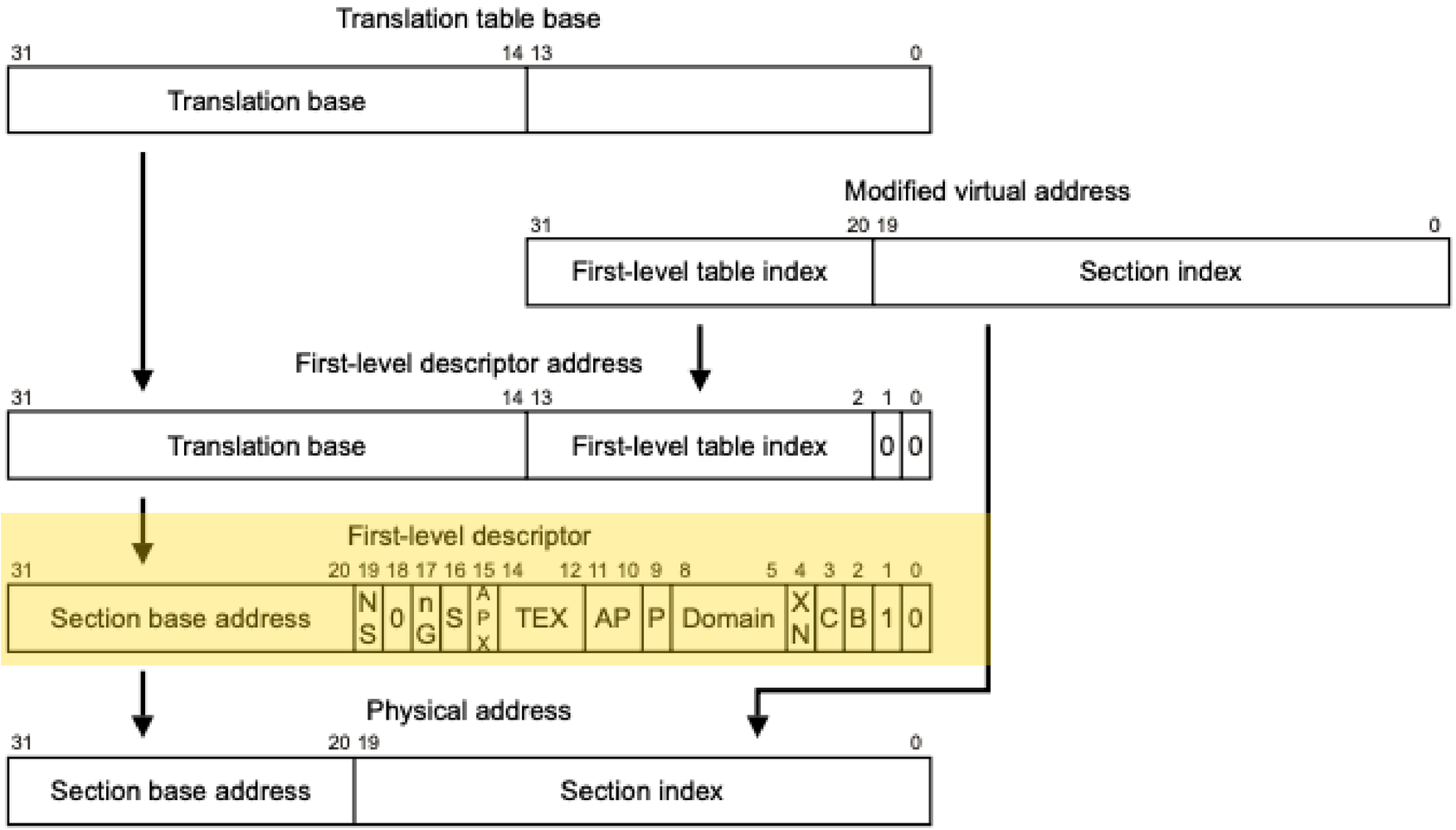
An aside: first level descriptros are **32-bit aligned** in memory (each entry is 4 bytes). Therefore, the lowest two bits should always be 0 (since 4 = 0b1**00**).



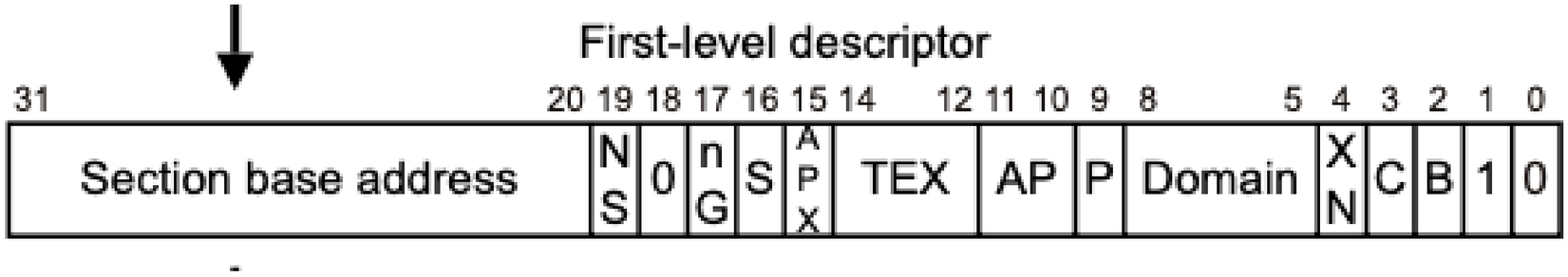
Once we have the **address** of the first level descriptor...



We can access it in memory!

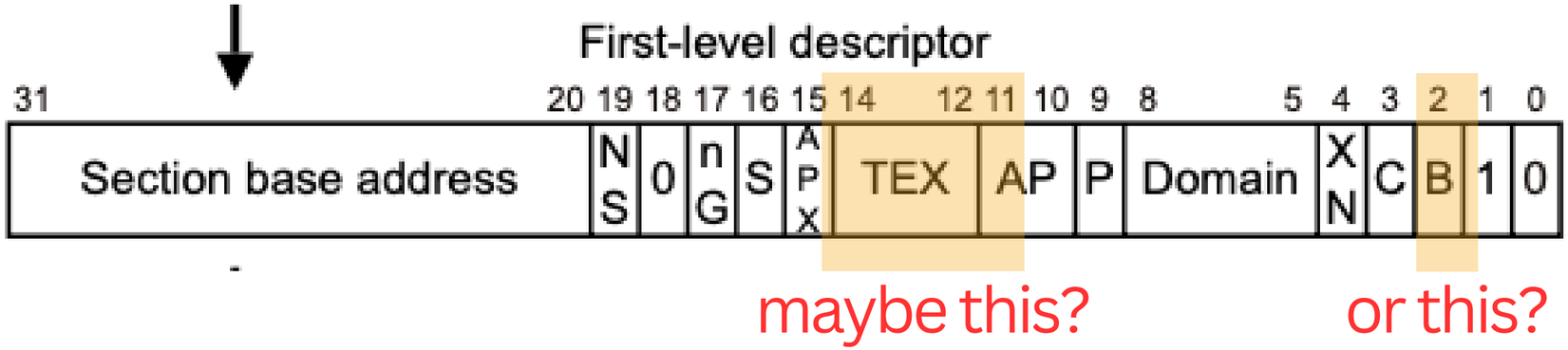


The **first level descriptor** holds information about how the virtual address is specifically mapped.



It holds things like domain permissions, memory attributes, etc. For this lab, you can keep it consistent with the entries specified in lab 17.

You'll also use this entry to help differentiate threads/entries that have the **same** virtual address for mappings).

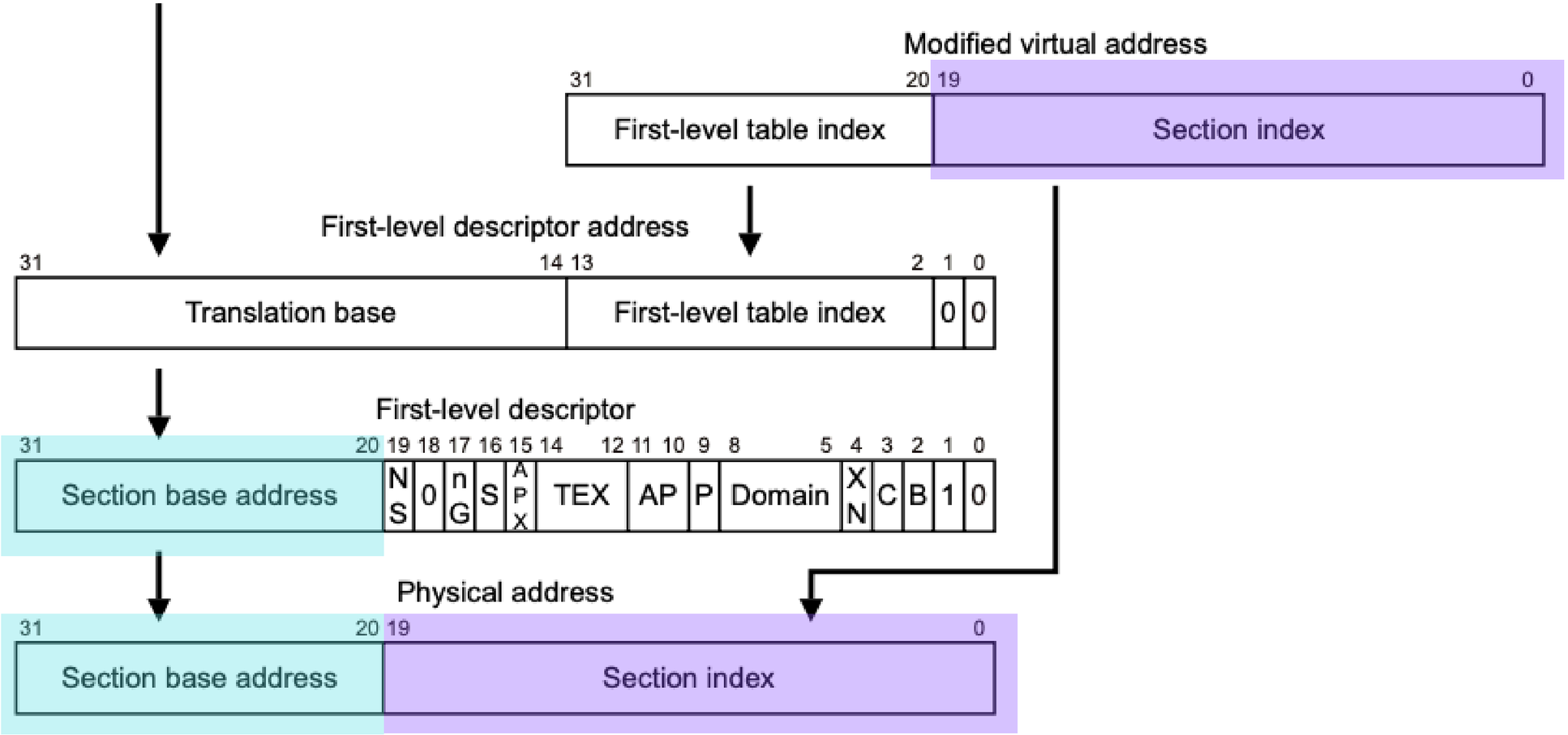


Dig into the first level descriptor fields to find where you could differentiate.

We care about the **section base address**, which will help us get the direct mapping of the physical address.

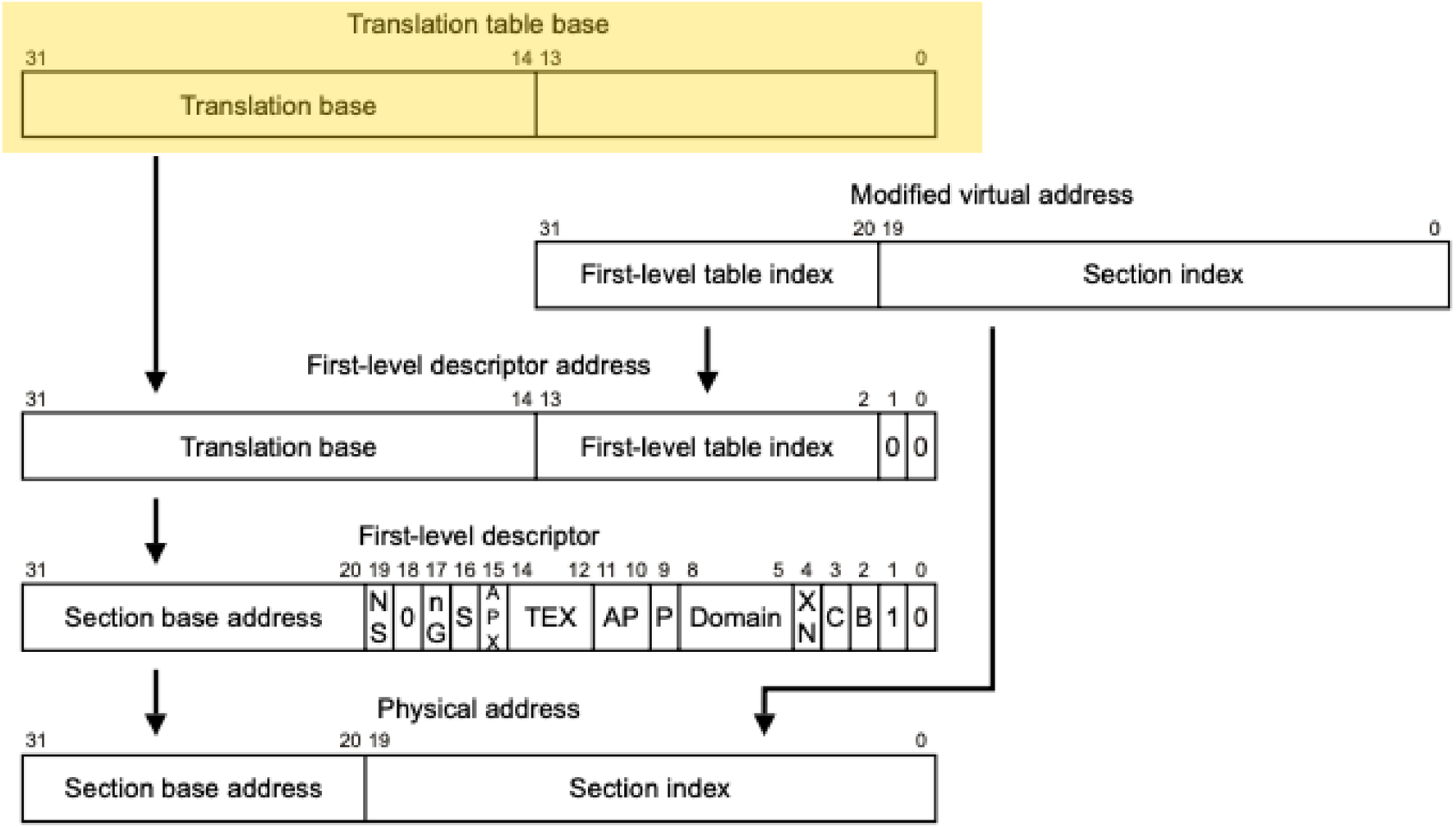
Store this!

To get the **physical address,** we'll need the virtual address and the first level descriptor like so:



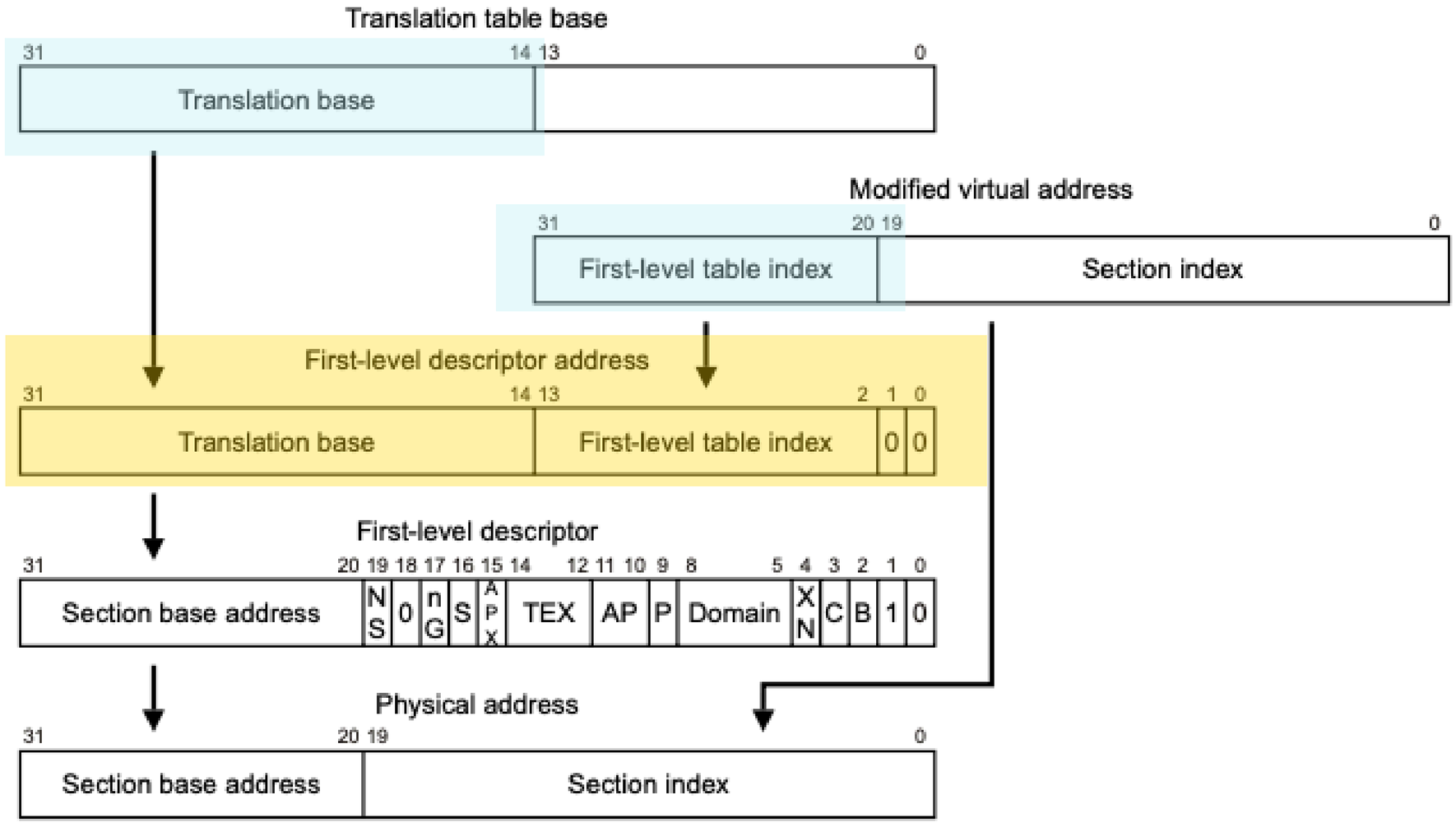
Yay! You've successfully:

Got the address of the active page table



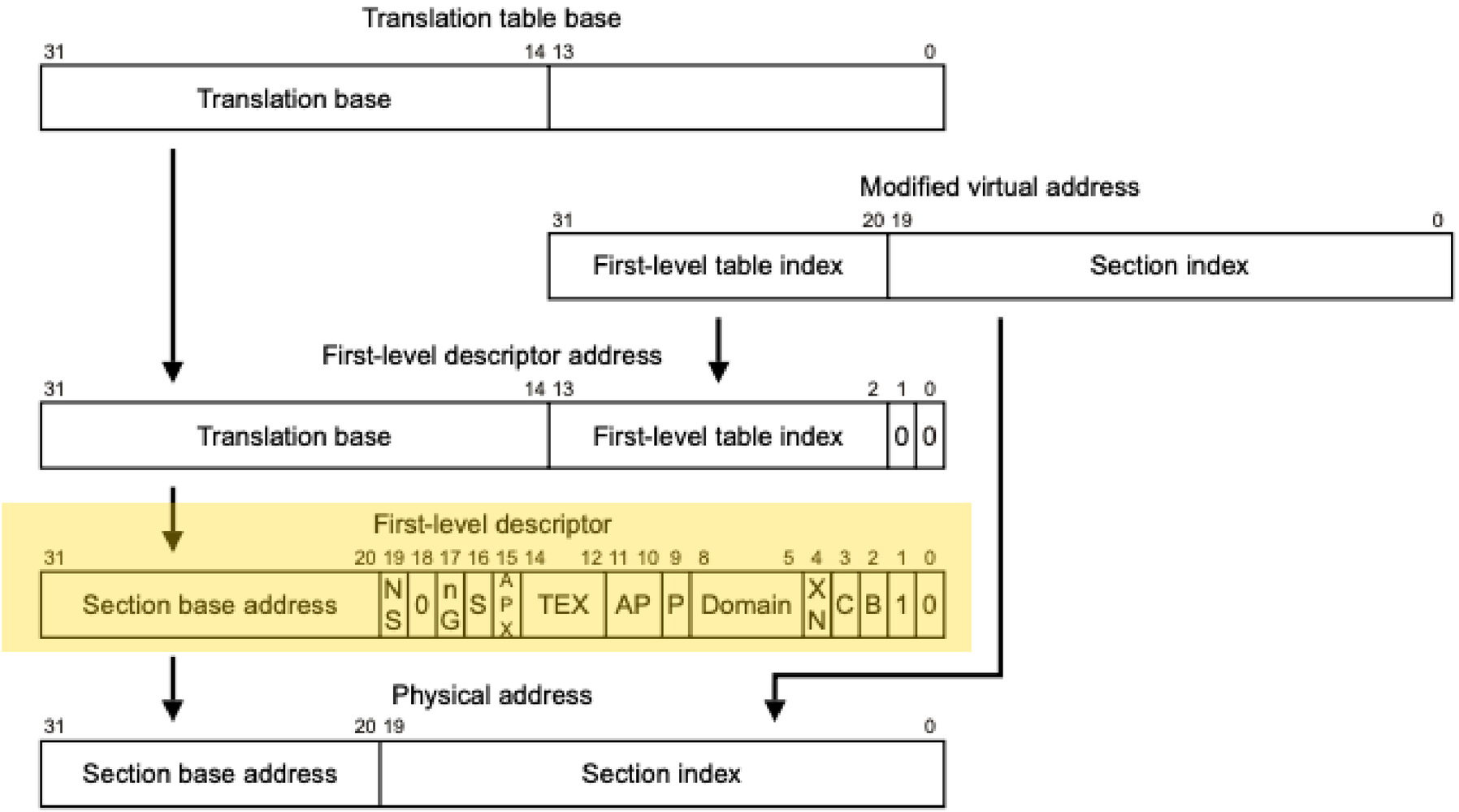
Yay! You've successfully:

Used information from the VA and TTBR0 to get the address of the entry



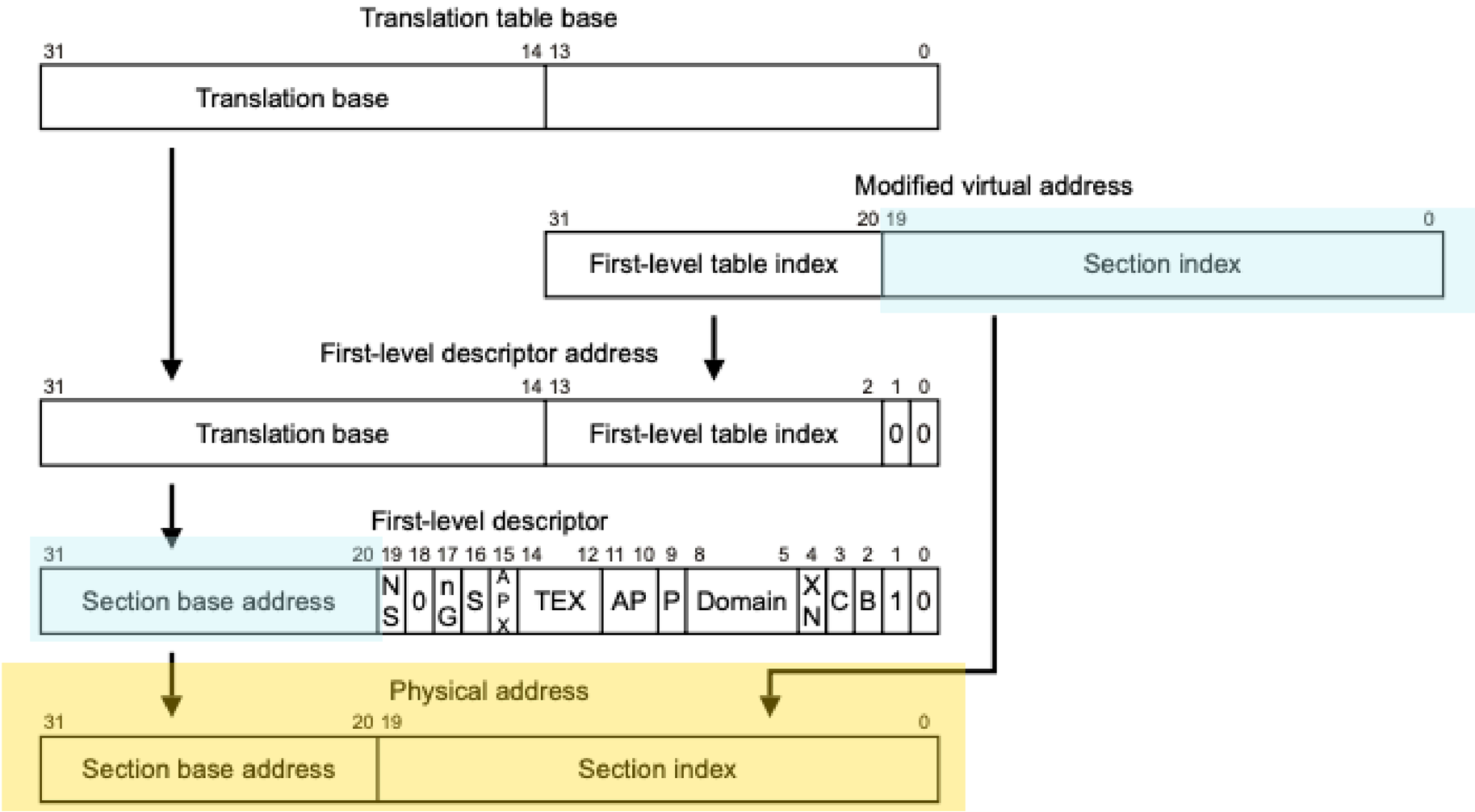
Yay! You've successfully:

Located the actual page table entry



Yay! You've successfully:

And used the VA and the entry to locate the desired PA



Reminder: why did we switch from TLB?

- **Scalability** – The TLB is a small cache; frequent swaps cause performance overhead.
- **Automatic TLB Management** – The processor naturally caches recent translations in the TLB.
- **Thread Isolation** – Each thread gets its own address space, preventing memory corruption between threads.

And... we can take a higher-level look at things:

The TLB is still involved, but we can trust the hardware/OS to handle it now! It will store entries accessed in page tables in the TLB automatically for easier access. We don’t have to manually do it.

And that’s all you need to understand for this next part of the lab!

Questions you should be able to answer

- What is the advantage of having a page table per thread instead of our previous TLB swapping approach?
- How do you get from a VA to a PA with the page table approach?